claude-windows / 96963dab-d38f-488c-af44-530ddbeefed1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbeefed1\subagents\workflows\wf_dd531439-5e0\agent-a6c41ee51c6374c7e.jsonl`
- SHA-256: `c4dbe480965e40362c2f78605c78193af48b3286c46d08b7c685349b6224838c`
- Source modified: `2026-07-08T06:32:44+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_dd531439-5e0`
- session_id: `96963dab-d38f-488c-af44-530ddbeefed1`

Transcript

1. user (2026-07-08T06:26:58.652Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).
To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::_md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-l4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.
DIMENSION: SECURITY / TRUST BOUNDARY. The core risk of win #2 is trusting a video_id instead of recomputing it. Attack the trust boundary:
- Can a CLIENT cause an ATTACKER-CONTROLLED video_id to reach the trusted path? Trace: /api/process body ? ProcessRequest (does it have a video_id field? it should NOT) ? submit_process_job (video_id = submit_time_video_id(input_ref) or "" ? server-derived, ignores any client value) ? claim_or_create_process_job(video_id=...) ? jobs.video_id ? _job_to_request ? _execute_processing(trusted_video_id=...). Is there ANY seam where a client value overrides the server-derived one?
- Cache poisoning / cross-tenant read: if a trusted id were wrong/spoofed, could it make the drain serve or write another video's data? Is the content-hash cache tenant-scoped or global, and does win #2 change that exposure vs today's submit-time cache probe (which ALREADY trusts submit_time_video_id)?
- Does trusting a GCS metadata md5 (vs re-hashing the downloaded bytes) weaken any integrity guarantee? GCS download verifies md5 by default ? is that relied on? What if a non-GCS backend (s3/gdrive) returned a video_id via submit_time_video_id? (Check: submit_time_video_id returns None for non-gcs ? confirm.)
- Is deployment.trust_submit_video_id=True a safe DEFAULT to ship on? Argue both sides.

2. assistant (2026-07-08T06:27:01.598Z)

I'll review the security/trust boundary of win #2. Let me start by examining the diff and the relevant code paths.

3. user (2026-07-08T06:27:04.654Z)

```
warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will
be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the
next time Git touches it
 backend/api/job_worker.py                          |   5 +
 backend/config/models.py                           |  10 +
 backend/orchestration.py                           |  51 ++++-
 .../L4_PRIMARY_TOPOLOGY.md                         | 225 ++++++++++++++++++++
 docs/COMPUTE_DECOUPLED_SERVING/README.md           |   2 +
 tests/test_config_deployment.py                    |   2 +
 tests/test_video_id_trust.py                       | 173 ++++++++++++++++
 7 files changed, 466 insertions(+), 2 deletions(-)
```

4. user (2026-07-08T06:27:05.919Z)

```
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
```

5. user (2026-07-08T06:27:14.127Z)

```
diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
                 db,
                 req,
                 progress=lambda stage: db.set_job_progress(job_id, stage),
+                # #524 win #2: hand the drain the submit-time md5 already on the job row
+                # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
+                # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+                # (local upload / non-gcs URI) ? the drain hashes as before.
+                trusted_video_id=job.get("video_id") or None,
             )
         if result.get("video_id"):
             db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win
#2).
+    # The chunked upload already streams the whole-file md5 (``backend/api/uploads.py``), and
the
+    # submit hook records it on the job row (``submit_time_video_id``, one gs:// metadata stat
? #484);
+    # that value is BYTE-IDENTICAL to ``compute_video_id`` (same bytes, same MD5) and is
ALREADY
+    # trusted for the submit-time cache probe (``backend/api/jobs.py``). When True (default)
the drain
+    # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
```

```
+    # (a local/appliance upload or a non-gcs direct-URI submit ? ``submit_time_video_id``
returns None)
+    # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+    # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+    trust_submit_video_id: bool = True


 class BillingConfig(BaseModel):
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 8b7851e..7769e4e 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
         return None


+def _looks_like_video_id(value: Optional[str]) -> bool:
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
+    ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2 trust
path:
+    a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
would
+    have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
+    it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
+    base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
+    return (
+        isinstance(value, str)
+        and len(value) == 32
+        and all(c in "0123456789abcdef" for c in value)
+    )
+
+
 def _cached_process_payload(
     video_id: str, validation_results, segments, requested_compute
 ) -> Dict[str, Any]:
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(


 def _execute_processing(
-    config=None, db=None, req=None, progress=None
+    config=None, db=None, req=None, progress=None, trusted_video_id=None
 ) -> Dict[str, Any]:
     """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
     test code that calls ``_execute_processing(req)`` still works.

     ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
     Raises on unexpected internal errors ? the caller records those as a job failure
     """
     import backend.main as _main  # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
         local_video = storage.acquire_local_input(
             req.video_path, getattr(config, "storage", None)
         )
```

```
-        video_id = _main.compute_video_id(local_video)
+        # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+        # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+        # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+        # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+        # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+        # is unchanged; only the redundant hash is skipped.
+        trust_submitted = bool(
+            getattr(
+                getattr(config, "deployment", None), "trust_submit_video_id", True
+            )
+        )
+        # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+        # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+        # silently coerce it into the DB key.
+        candidate = trusted_video_id or ""
+        if trust_submitted and _looks_like_video_id(candidate):
+            video_id = candidate
+            logger.info(
+                "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+                video_id,
+            )
+        else:
+            video_id = _main.compute_video_id(local_video)
     except Exception:
         storage.cleanup_acquired_local_input(
             req.video_path, local_video, getattr(config, "storage", None)
```

6. user (2026-07-08T06:27:19.173Z)

```
1       """#524 win #2 ? trust the submit-time ``video_id`` instead of re-hashing on the drain.
2
3       The chunked upload streams the whole-file MD5 while it uploads
(``backend/api/uploads.py``), and the
4       submit hook records that same md5 on the job row (``submit_time_video_id``, #484 ? one
gs:// metadata
5       stat, converted to the identical HEX form ``compute_video_id`` produces). So when the
drain fetches
6       the clip to scratch it already KNOWS the video_id and need not re-hash the (often multi-
GB) file.
7
8       These tests pin the behaviour: with a well-formed trusted id and the knob on (default),
the drain
9       SKIPS ``compute_video_id`` and uses the trusted id verbatim; with the id absent,
malformed, or the
10      knob off, it hashes exactly as before. A spy on ``compute_video_id`` (counting
invocations) is the
11      proof ? mirrors ``tests/test_validation_cache.py``'s stubbed-suite pattern (fully
offline, no decode).
12      """
13
14      import backend.main as m
15      from backend.config.models import AppConfig, DeploymentConfig
16      from backend.infrastructure.database import Database
17      from backend.orchestration import _looks_like_video_id
18      from backend.pipeline.validators.base import ValidationResult
19
20      _VALID_ID = "a" * 32  # a well-formed 32-char lowercase-hex video_id
```

```
    _HASH_SENTINEL = "b" * 32  # what a real compute_video_id() would return (spied)


    class _HashSpy:
        """Stands in for ``compute_video_id`` and counts real invocations. A TRUST hit must
NOT call this
        ? ``calls == 0`` is the proof the drain skipped the re-hash."""

        def __init__(self, ret=_HASH_SENTINEL):
            self._ret = ret
            self.calls = 0

        def __call__(self, *a, **k):
            self.calls += 1
            return self._ret


    class _OkSeg:
        """A segmenter that writes one play row (validation is stubbed to pass, so it's
reached)."""

        def __init__(self, db_, cfg_):
            self.db = db_

        def process_video(self, vid, path, *a, **k):
            sid = self.db.add_play_segment(vid, 0.0, 5.0)
            return [
                {"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0, "metadata":
{}}
            ]


    def _pass_validators(*a, **k):
        return [ValidationResult(validator_name="x", passed=True, message="ok",
details={})], {}


    def _setup(tmp_path, monkeypatch, *, cfg=None):
        """Wire module globals + a temp-file DB + a hash spy + stubbed validators/segmenter.
        Returns ``(hash_spy, video_path)``. A bare/local ``video_path`` makes
``acquire_local_input`` an
        identity (no fetch), so the ONLY video_id source is the trusted id or the spied
hash."""
        db = Database(str(tmp_path / "t.db"))
        monkeypatch.setattr(m, "db_instance", db)
        monkeypatch.setattr(m, "global_config", cfg or AppConfig())
        spy = _HashSpy()
        monkeypatch.setattr(m, "compute_video_id", spy)
        monkeypatch.setattr(m.registry, "run_all_with_context", _pass_validators)
        monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _OkSeg)
        video = tmp_path / "v.mp4"
        video.write_bytes(b"some-bytes")
        return spy, str(video)


    def _process(video, *, trusted):
        # config resolves from m.global_config (set by _setup) via the backward-compat shim.
        return m._execute_processing(
            m.ProcessRequest(video_path=video, segmenter="motion"),
            trusted_video_id=trusted,
        )


    # --- _looks_like_video_id (the format guard)
-----------------------------------------------------
```

```
79
80
81      def test_looks_like_video_id_accepts_only_32_char_lowercase_hex():
82          assert _looks_like_video_id("a" * 32)
83          assert _looks_like_video_id("0123456789abcdef0123456789abcdef")
84          # rejects: wrong length, uppercase, non-hex, non-str, empty, None
85          assert not _looks_like_video_id("a" * 31)
86          assert not _looks_like_video_id("a" * 33)
87          assert not _looks_like_video_id("A" * 32)  # helper is strict-lowercase (callers
.lower() first)
88          assert not _looks_like_video_id("g" * 32)
89          assert not _looks_like_video_id("not-a-hash")
90          assert not _looks_like_video_id("")
91          assert not _looks_like_video_id(None)
92          assert not _looks_like_video_id(12345678901234567890123456789012)  # int, not str
93
94
95      # --- the drain: trust vs re-hash
----------------------------------------------------------------------
96
97
98      def test_trusts_valid_submit_id_and_skips_rehash(tmp_path, monkeypatch, caplog):
99          """A well-formed trusted id + the default knob ? the drain reuses it and does NOT
call the hash."""
100         spy, video = _setup(tmp_path, monkeypatch)
101         with caplog.at_level("INFO", logger="backend.orchestration"):
102             out = _process(video, trusted=_VALID_ID)
103         assert out["video_id"] == _VALID_ID
104         assert spy.calls == 0  # the whole point: no re-hash
105         assert any("trusting submit-time video_id" in r.message for r in caplog.records)
106
107
108     def test_hashes_when_no_trusted_id(tmp_path, monkeypatch):
109         """No trusted id (local upload / non-gcs URI ? submit_time_video_id was None) ? hash
as before."""
110         spy, video = _setup(tmp_path, monkeypatch)
111         out = _process(video, trusted=None)
112         assert out["video_id"] == _HASH_SENTINEL
113         assert spy.calls == 1
114
115
116     def test_ignores_trusted_id_when_knob_off(tmp_path, monkeypatch):
117         """deployment.trust_submit_video_id=False forces the re-hash even with a valid
trusted id
118         (belt-and-suspenders rollback, no redeploy)."""
119         cfg = AppConfig(deployment=DeploymentConfig(trust_submit_video_id=False))
120         spy, video = _setup(tmp_path, monkeypatch, cfg=cfg)
121         out = _process(video, trusted=_VALID_ID)
122         assert out["video_id"] == _HASH_SENTINEL
123         assert spy.calls == 1
124
125
126     def test_ignores_malformed_trusted_id_and_falls_back_to_hash(tmp_path, monkeypatch):
127         """A malformed / wrong-length / wrong-encoding trusted id can never become the DB
key ?
128         it degrades to a real hash."""
129         spy, video = _setup(tmp_path, monkeypatch)
130         for bad in ("not-a-valid-id", "a" * 31, "A" * 32, ""):
131             spy.calls = 0
132             out = _process(video, trusted=bad)
133             assert out["video_id"] == _HASH_SENTINEL, bad
134             assert spy.calls == 1, bad
135
136
137     # --- the wiring: the drain hands _execute_processing the job row's video_id
```

```
------------------------
138
139
140     def test_run_claimed_job_threads_job_video_id_as_trusted(tmp_path, monkeypatch):
141         """``_run_claimed_job`` must pass the job row's persisted video_id
(submit_time_video_id, #484) to
142         ``_execute_processing`` as ``trusted_video_id`` ? the wiring that makes win #2 fire
on the drain."""
143         db = Database(str(tmp_path / "jobs.db"))
144         monkeypatch.setattr(m, "db_instance", db)
145         monkeypatch.setattr(m, "global_config", AppConfig())
146         seen = {}
147
148         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
149             seen["trusted"] = trusted_video_id
150             return {"status": "success", "video_id": _VALID_ID, "play_segments": []}
151
152         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
153         db.create_job("pj", "gs://b/clip.mp4")
154         db.set_job_video_id("pj", _VALID_ID)
155         m._run_claimed_job(db.get_job("pj"), m.global_config, m.db_instance)
156         assert seen["trusted"] == _VALID_ID
157
158
159     def test_run_claimed_job_passes_none_when_job_has_no_video_id(tmp_path, monkeypatch):
160         """A job with no known video_id (local upload) threads None ? the drain hashes
(fallback)."""
161         db = Database(str(tmp_path / "jobs.db"))
162         monkeypatch.setattr(m, "db_instance", db)
163         monkeypatch.setattr(m, "global_config", AppConfig())
164         seen = {}
165
166         def _spy_exec(config, db_, req, progress=None, trusted_video_id=None):
167             seen["trusted"] = trusted_video_id
168             return {"status": "success", "video_id": "x", "play_segments": []}
169
170         monkeypatch.setattr(m, "_execute_processing", _spy_exec)
171         db.create_job("pj2", "/local/v.mp4")  # no video_id set
172         m._run_claimed_job(db.get_job("pj2"), m.global_config, m.db_instance)
173         assert seen["trusted"] is None
174
```

7. user (2026-07-08T06:27:20.028Z)

```
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
```

```
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception:  # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236                 getattr(input_ref, "uri", input_ref),
237                 exc_info=True,
238             )
239             return None
240
241
242     def _looks_like_video_id(value: Optional[str]) -> bool:
243         """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5
that
244         ``compute_video_id`` produces (`backend/utils/hashing.py`). Guards the #524 win-#2
trust path:
245         a trusted id is only reused in place of a re-hash when it has the EXACT shape the
drain would
246         have computed, so a malformed / truncated / wrong-encoding id can never become the
DB key ?
247         it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the
gs://
248         base64 md5 to this hex form (#484, `backend/storage/gcs.py::_md5_hex`)."""
249         return (
250             isinstance(value, str)
251             and len(value) == 32
252             and all(c in "0123456789abcdef" for c in value)
253         )
254
255
256     def _cached_process_payload(
257         video_id: str, validation_results, segments, requested_compute
258     ) -> Dict[str, Any]:
259         """The /api/process result for a video whose detection already exists ? ONE shape
for
260         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
261         return {
262             "video_id": video_id,
263             "status": "success",
264             "message": (
265                 f"Reused {len(segments)} cached play segments. "
266                 "Pass force=true to reprocess this video."
267             ),
268             "results": validation_results or [],
269             "play_segments": segments,
270             "cached": True,
271             "compute": {"path": "cached", "requested": requested_compute},
272         }
273
```

```
274
275     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
276         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
277         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
278         ref = storage.parse_uri(uri)
279         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
280             local = storage.fetch(
281                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
282             )
283             with open(local, encoding="utf-8") as f:
284                 loaded = json.load(f)
285         return loaded if isinstance(loaded, dict) else None
286
287
288     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
289     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
290     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
291     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
292     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
293     _REHYDRATE_LOCKS_GUARD = threading.Lock()
294
295
296     def _rehydrate_lock(video_id: str) -> threading.Lock:
297         with _REHYDRATE_LOCKS_GUARD:
298             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
299
300
301     def probe_process_cache(db, cfg, video_id: str) -> bool:
302         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
303
304         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
305         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
306         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
307         errors report False (? the normal dispatch path with its guards)."""
308         try:
309             existing = db.get_video(video_id)
310             if (
311                 existing
312                 and existing.get("status") == "processed"
313                 and db.query_play_segments(video_id, {})
314             ):
315                 return True
316             sc = _storage_settings(cfg)
317             if not sc or (sc.get("backend") or "local") == "local":
318                 return False
319             output_root = (sc.get("output_root") or "").rstrip("/")
```

8. assistant (2026-07-08T06:27:24.714Z)

Now let me trace the submit path and the ProcessRequest model.

9. user (2026-07-08T06:27:28.115Z)

```
backend/orchestration.py:220:def submit_time_video_id(input_ref, cfg) -> Optional[str]:
backend/orchestration.py:247:    it degrades to a real hash instead. ``submit_time_video_id``
already normalizes the gs://
backend/orchestration.py:735:    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ?
byte-identical to
backend/orchestration.py:787:        # lowercase hex, per compute_video_id /
```

```
submit_time_video_id). No normalization ? a
backend/api/jobs.py:33:    ``db.claim_or_create_process_job`` (ONE write-locked op) so two
concurrent identical submits ? a
backend/api/jobs.py:41:    claimed = db.claim_or_create_process_job(
backend/api/jobs.py:74:    from backend import orchestration  # call-time seam (mirrors
submit_process_job)
backend/api/jobs.py:80:def submit_process_job(
backend/api/jobs.py:98:    video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
backend/api/job_worker.py:122:def _job_to_request(job: Dict[str, Any]):
backend/api/job_worker.py:170:            req = _main._job_to_request(job)
backend/api/job_worker.py:177:                # (``submit_time_video_id``, #484 ? server-
derived, byte-identical to
backend/api/job_worker.py:183:            db.set_job_video_id(job_id, result["video_id"])
```

10. user (2026-07-08T06:27:30.412Z)

```
1       """Job-list endpoint (the processing-queue view) as its own router ? kept OUT of the
backend.main
2       god-module per the P24 route ceiling (see tests/test_main_module_budget.py; pattern:
backend/api/uploads.py).
3
4       GET /api/jobs is tenant-scoped and status-filtered so a client can discover in-flight
work it did NOT
5       submit (a page reload, a new tab, another device) and show its status ? which also stops
a tester who
6       can't see their running job from re-submitting the same video into a second, PAID GPU
run (#448). The
7       per-submit dedup that makes that safe server-side lives inline in
backend.main.process_video.
8
9       Reads the live DB + config off ``request.app.state`` (set in backend.main.create_app),
so it needs no
10      module-global wiring and stays trivially testable via the standard
TestClient(create_app(cfg, db)).
11      """
12
13      import tempfile
14      from typing import Any, List, Optional, Tuple
15
16      from fastapi import APIRouter, HTTPException, Request
17      from fastapi.responses import JSONResponse
18
19      from backend.api import (
20          auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
21      )
22      from backend.utils.freespace import (
23          require_free_bytes,  # HTTP-507 scratch guard (P2, D11)
24      )
25
26      router = APIRouter()
27
28
29      def claim_process_job(
30          db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
31      ) -> Optional[Tuple[bool, Optional[JSONResponse]]]:
32          """ATOMIC idempotent-submit hook for backend.main.process_video (PR #481 review):
delegates to
33          ``db.claim_or_create_process_job`` (ONE write-locked op) so two concurrent identical
submits ? a
34          double-click / retry ? can't both create a job and start a second PAID GPU run.
Returns
35          ``(created, dedup_response)``: when ``created`` is True the caller dispatches; when
False the caller
36          just returns ``dedup_response`` (a 202 pointing at the already-active job). ``None``
on DB failure.
```

```
        ``req.force`` opts out (deliberate reprocess). Keyed on (owner_id, video_path) ? the
input isn't
        hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
        the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
        can re-associate the job with its bucket outputs (CP-rebuild §4.2); "" ? the drain
fills it."""
        claimed = db.claim_or_create_process_job(
            job_id,
            req.video_path,
            segmenter=req.segmenter,
            player_pool=req.player_pool,
            max_frames=req.max_frames,
            owner_id=owner_id,
            locale=req.locale,
            compute=req.compute,
            force=req.force,
            video_id=video_id,
        )
        if claimed is None:
            return None
        if claimed["created"]:
            return (True, None)
        return (
            False,
            JSONResponse(
                status_code=202,
                content={
                    "job_id": claimed["job_id"],
                    "status": claimed["status"],
                    "poll": f"/api/jobs/{claimed['job_id']}",
                    "deduplicated": True,
                },
            ),
        )


    def _fetch_scratch_guard(cfg: Any, input_ref: Any) -> None:
        """P2/D11 507 fast-fail for a remote input the DRAIN will fetch to local scratch
        (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the
copy."""
        from backend import orchestration  # call-time seam (mirrors submit_process_job)

        need = orchestration._remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
        require_free_bytes(tempfile.gettempdir(), need, stage="fetch")


    def submit_process_job(
        db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
    ) -> Optional[Tuple[bool, JSONResponse]]:
        """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
one hook.

        Order matters (#492 review):
        1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
effects.
        2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
           fetches to local scratch, so low temp space must not reject it (review blocker
2).
        3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
           re-hydrate side effect, so the first request owns the cache completion (blocker
1).
        4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
```

```
serialized
92             per video_id) and completes the job instantly: no drain, no GPU dispatch.
93
94          Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
``None`` ?
95          DB failure (the caller 500s)."""
96          from backend import orchestration  # call-time seam: tests patch orchestration attrs
97
98          video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99          probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100             db, cfg, video_id
101         )
102         if input_ref.backend != "local" and not probed:
103             _fetch_scratch_guard(cfg, input_ref)  # raises 507 pre-claim, exactly as before
104         outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
105         if outcome is None:
106             return None
107         created, dedup_response = outcome
108         if not created:
109             assert dedup_response is not None  # claim contract: not-created ? dedup
response
110             return (False, dedup_response)
111         if probed:
112             cached = orchestration.cached_process_result(
113                 db, cfg, video_id, req.video_path, requested_compute=req.compute
114             )
115             if cached is not None:
116                 db.mark_job_done(job_id, cached)
117                 return (
118                     False,
119                     JSONResponse(
120                         status_code=202,
121                         content={
122                             "job_id": job_id,
123                             "status": "done",
124                             "poll": f"/api/jobs/{job_id}",
125                             "cached": True,
126                         },
127                     ),
128                 )
129             # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
130             # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
131             # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
132             # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133             if input_ref.backend != "local":
134                 try:
135                     _fetch_scratch_guard(cfg, input_ref)
136                 except HTTPException:
137                     db.mark_job_failed(
138                         job_id,
139                         "cache re-hydrate failed and local scratch cannot hold a fetch of "
140                         "the source; free disk space or retry later",
141                     )
142                     raise
143         return (
144             True,
145             JSONResponse(
146                 status_code=202,
147                 content={
148                     "job_id": job_id,
149                     "status": "queued",
150                     "poll": f"/api/jobs/{job_id}",
```

```
151                 },
152             ),
153         )
154 
155     # Execution states a job can be in. ACTIVE = still doing (or about to do) work; the
queue view defaults
156     # to these. done/failed are terminal and only surface under ?status=all (or an explicit
single status).
157     ACTIVE_JOB_STATES = ("queued", "running", "waiting")
158     _LISTABLE_JOB_STATES = frozenset(ACTIVE_JOB_STATES) | {"done", "failed"}
159 
160 
161     @router.get("/api/jobs")
162     def list_jobs(request: Request, status: str = "active"):
163         """The caller's jobs (tenant-scoped), newest first. ``status=active`` (default) ?
164         queued|running|waiting; ``status=all`` ? recent jobs of any state; a single explicit
state is also
165         accepted (anything unknown degrades to the active queue ? never a raw 500). Light
projection: the
166         heavy per-job ``play_segments`` payload is dropped; ``result_status`` carries the
terminal verdict."""
167         try:
168             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
169         except edge_auth.EdgeAuthError as e:
170             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
171         if status == "all":
172             statuses: Optional[List[str]] = None
173         elif status in _LISTABLE_JOB_STATES:
174             statuses = [status]
175         else:  # "active" and any unknown value ? the active queue
176             statuses = list(ACTIVE_JOB_STATES)
177         rows = request.app.state.db.list_jobs(owner_id, statuses=statuses, limit=50)
178         return {
179             "jobs": [
180                 {
181                     "job_id": j["id"],
182                     "kind": j.get("kind") or "process",
183                     "status": j["status"],
184                     "progress": j.get("progress"),
185                     "video_id": j.get("video_id"),
186                     "source": j.get("video_path"),
187                     "compute": j.get("compute"),
188                     "result_status": j.get("result_status"),
189                     "error": j.get("error"),
190                     "created_at": j.get("created_at"),
191                     "started_at": j.get("started_at"),
192                     "finished_at": j.get("finished_at"),
193                 }
194                 for j in rows
195             ]
196         }
197 
```

11. assistant (2026-07-08T06:27:33.250Z)

Now let me look at the ProcessRequest model, the process_video endpoint, and `_job_to_request`.

12. user (2026-07-08T06:27:34.852Z)

```
110 
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115 
```

```
        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main

        raw_params = job.get("params")
        params = (
            raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
        )
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
            # v8 compute-picker: the worker runs _execute_processing ?
_maybe_dispatch_remote, which
            # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
            # the job row for analytics). NULL ? server default = pre-picker behaviour.
            compute=job.get("compute"),
            force=bool(params.get("force")),
        )


    def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
        """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
        terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).

        Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
        the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
        result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
        job self-scheduled and will be re-claimed when `next_poll_at` is due.
        """
        import backend.main as _main

        job_id = job["id"]
        # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
        # original /api/process pipeline). Both record their terminal state the same way;
only process
        # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
        is_compile = job.get("kind") == "compile"
        try:
            if is_compile:
                result = _main._execute_compile(
                    config,
                    db,
                    job,
                    progress=lambda stage: db.set_job_progress(job_id, stage),
                )
            else:
                req = _main._job_to_request(job)
                result = _main._execute_processing(
                    config,
```

```
173                     db,
174                     req,
175                     progress=lambda stage: db.set_job_progress(job_id, stage),
176                     # #524 win #2: hand the drain the submit-time md5 already on the job row
177                     # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
178                     # compute_video_id) so it can skip re-hashing the fetched clip. "" ?
unknown
179                     # (local upload / non-gcs URI) ? the drain hashes as before.
180                     trusted_video_id=job.get("video_id") or None,
181                 )
182             if result.get("video_id"):
183                 db.set_job_video_id(job_id, result["video_id"])
184             db.mark_job_done(job_id, result)
185             if not is_compile:
186                 _maybe_record_job_cost(
187                     job, result, config, db
188                 )  # M8 cost ledger (default-OFF; process jobs only)
189                 _maybe_run_flywheel(
190                     job, result, config
191                 )  # M11 data-flywheel (process jobs only)
192         except _main.JobWaiting as w:
193             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
194             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
195             db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
196         except Exception as e:
197             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
198             db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
199
```

13. user (2026-07-08T06:27:37.101Z)

```
94:from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
258:class ProcessRequest(BaseModel):
277:    # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
verbatim.
278:    video_id: str
289:    # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
297:    video_id: str
305:    video_id: str
439:    "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
448:        "video_id": row.get("id"),
461:    (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
482:    does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
512:    """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
549:def _source_cache_lock(video_id: str) -> threading.Lock:
551:        return _source_cache_locks.setdefault(video_id, threading.Lock())
554:def _source_cache_path(video_id: str, input_ref, cfg) -> str:
559:    name = safe_output_filename(f"{video_id}{ext}")
563:def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:    cache_path = _source_cache_path(video_id, input_ref, cfg)
568:    lock = _source_cache_lock(video_id)
591:def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
596:        if video_id in _proxy_inflight:
598:        _proxy_inflight.add(video_id)
605:            src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608:        logger.warning("proxy: warm failed for %s: %s", video_id, e)
611:            _proxy_inflight.discard(video_id)
614:@app.get("/api/source/{video_id}")
615:def get_source(video_id: str, request: Request):
621:    rec = request.app.state.db.get_video(video_id)
628:        proxy_name = safe_output_filename(f"{video_id}.mp4")
```

```
655:            local = _cached_remote_source(video_id, input_ref, cfg)
667:        target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
677:    ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?
so it joins
681:    Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
691:        name = safe_output_filename(f"{req.video_id}.rallies.csv")
693:        raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
707:        "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
720:def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721:    """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``."""
722:    return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
753:    MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
```

14. assistant (2026-07-08T06:27:39.956Z)

Let me look at the ProcessRequest model definition and the /api/process endpoint that calls submit_process_job.

15. user (2026-07-08T06:27:41.536Z)

```
258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269         compute: Optional[str] = None
270         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271         # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
272         force: bool = False
273
274
275     class AnnotationsRequest(BaseModel):
276         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278         video_id: str
279         csv: str
280
281
282     class AssetRequest(BaseModel):
283         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
284         #   uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
285         #                   local working copy for the run), OR
286         #   source_uri    ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
287         #                   registered as-is, no copy (D14b).
288         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
```

```
is keyed by
289         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
290         uploaded_path: Optional[str] = None
291         source_uri: Optional[str] = None
292         medium_uri: str
```

16. user (2026-07-08T06:27:44.171Z)

```
554:def _source_cache_path(video_id: str, input_ref, cfg) -> str:
555:    ext = os.path.splitext(input_ref.name or "")[1].lower()
563:def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:    cache_path = _source_cache_path(video_id, input_ref, cfg)
574:        need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
580:            storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
639:        input_ref = storage.resolve_input_ref(
651:    if input_ref.backend == "local":
652:        local = input_ref.key
655:            local = _cached_remote_source(video_id, input_ref, cfg)
684:        edge_auth.resolve_tenant(request.headers, request.app.state.config)
756:        owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
786:            src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
798:            src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
834:                medium_ref = storage.parse_uri(req.medium_uri)
884:            owner_id,
905:def process_video(req: ProcessRequest, request: Request):
913:    # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-
user,
915:    # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
917:        owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
935:        input_ref = storage.resolve_input_ref(
941:    # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI string. A
946:    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
950:    # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
964:    outcome = jobs_api.submit_process_job(
965:        request.app.state.db, request.app.state.config, job_id, req, owner_id, input_ref
1110:        owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1126:    # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
1134:        owner_id=owner_id,
1499:            input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1503:        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

17. user (2026-07-08T06:27:48.299Z)

```
905     def process_video(req: ProcessRequest, request: Request):
906         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
907
908         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
909         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
910         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
911         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
912         """
913         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
914         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
915         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916         try:
917             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918         except edge_auth.EdgeAuthError as e:
919             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
```

```
920
921         allowed_root = getattr(
922             getattr(request.app.state.config, "security", None), "allowed_video_root", None
923         )
924         # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925         # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926         allowed_uri_root = getattr(
927             getattr(request.app.state.config, "storage", None), "input_root", None
928         )
929         try:
930             validate_input_path(
931                 req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
932             )
933             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935             input_ref = storage.resolve_input_ref(
936                 req.video_path, getattr(request.app.state.config, "storage", None)
937             )
938         except ValueError as e:
939             raise HTTPException(status_code=400, detail=str(e)) from e
940         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
944         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945         # rejects a non-local URI as out-of-root.
946         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947             raise HTTPException(
948                 status_code=404, detail=f"Video file not found at '{req.video_path}'"
949             )
950         # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
951         # the cache probe (#492 review): a cached gs:// result never fetches to local
scratch, so
952         # low temp space must not 507 it; a cache MISS still fails fast pre-claim exactly as
before.
953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
961         # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
962         # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
963         job_id = uuid.uuid4().hex
964         outcome = jobs_api.submit_process_job(
965             request.app.state.db, request.app.state.config, job_id, req, owner_id, input_ref
966         )
967         if outcome is None:
968             raise HTTPException(status_code=500, detail="Failed to persist job record.")
969         queued, response = outcome
970         if queued:
```

```
971             _get_executor().submit(
972                 _drain_due_jobs, request.app.state.config, request.app.state.db
973             )
974         return response
975
976
977     @app.get("/api/jobs/{job_id}/cost")
978     def get_job_cost(job_id: str, request: Request):
979         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
980         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
981         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
982         cost = request.app.state.db.get_job_cost(job_id)
983         if cost is None:
984             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
```

18. assistant (2026-07-08T06:27:54.264Z)

Let me look at the GCS `_md5_hex` normalization and the stat/md5 path.

19. user (2026-07-08T06:27:57.568Z)

```
backend/storage/gcs.py:24:def _md5_hex(md5_b64: "str | None") -> Optional[str]:
backend/storage/gcs.py:25:    """GCS reports ``md5Hash`` as base64 of the raw digest;
``compute_video_id`` (the DB key)
backend/storage/gcs.py:27:    have no md5Hash ? None; a malformed value is treated as absent,
never raised."""
backend/storage/gcs.py:28:    if not md5_b64:
backend/storage/gcs.py:31:        raw = base64.b64decode(md5_b64, validate=True)
backend/storage/gcs.py:75:    def stat(self, ref) -> StorageStat:
backend/storage/gcs.py:83:            md5=_md5_hex(getattr(b, "md5_hash", None)),
backend/storage/base.py:47:    def stat(self, ref: StorageRef) -> StorageStat: ...
```

20. user (2026-07-08T06:28:00.604Z)

```
24      def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25          """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26          is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27          have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28          if not md5_b64:
29              return None
30          try:
31              raw = base64.b64decode(md5_b64, validate=True)
32          except (binascii.Error, ValueError):
33              return None
34          return raw.hex() if len(raw) == 16 else None
35
36
37      class GCSStorage(StorageBackend):
38          def __init__(
39              self,
40              *,
41              client: Any = None,
42              project: Optional[str] = None,
43              signed_url_ttl: int = 3600,
44              **_ignored,
45          ) -> None:
46              self._ttl = signed_url_ttl
47              if client is not None:
48                  self._client = client
49                  return
```

```
50              storage_module: Any = import_module("google.cloud.storage")
51              self._client = storage_module.Client(project=project)
52
53          def _blob(self, ref: StorageRef):
54              return self._client.bucket(ref.bucket).blob(ref.key)
55
56          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57              if dst is None:
58                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59              import os
60
61              dst = str(dst)
62              if os.path.exists(dst) and not overwrite:
63                  return dst
64              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
65              self._blob(ref).download_to_filename(dst)
66              return dst
67
68          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
69              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
70              return ref
71
72          def exists(self, ref) -> bool:
73              return bool(self._blob(ref).exists())
74
75          def stat(self, ref) -> StorageStat:
76              b = self._blob(ref)
77              b.reload()
78              return StorageStat(
79                  size=int(b.size or 0),
80                  modified=b.updated,
81                  etag=b.etag,
82                  content_type=b.content_type,
83                  md5=_md5_hex(getattr(b, "md5_hash", None)),
84              )
85
86          def list(self, prefix) -> Iterator[StorageRef]:
87              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
88                  yield StorageRef(
89                      "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
90                  )
91
92          def delete(self, ref) -> None:
93              self._blob(ref).delete()
```

21. assistant (2026-07-08T06:29:04.980Z)

Let me examine the drain's fetch path (`acquire_local_input`) and check whether GCS download verification / generation pinning applies.

22. user (2026-07-08T06:29:08.379Z)

```
backend/storage/__init__.py:77:def acquire_local_input(
backend/storage/__init__.py:170:def fetch(
backend/storage/gcs.py:65:        self._blob(ref).download_to_filename(dst)
backend/storage/__init__.py:77:def acquire_local_input(
backend/storage/__init__.py:170:def fetch(
backend/storage/gcs.py:65:        self._blob(ref).download_to_filename(dst)
```

23. user (2026-07-08T06:29:10.885Z)

```
77      def acquire_local_input(
78          raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
79      ) -> str:
```

```
        """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.

        Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
        byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
        non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
        path returned. The caller owns successful remote scratch copies; use
        ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
        sc = _storage_dict(storage_config)
        ref = resolve_input_ref(raw, sc)
        if ref.backend == "local":
            return (
                ref.key
            )  # identity ? no temp, no copy, byte-identical to passing the path directly
        scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
        try:
            return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
        except Exception:
            shutil.rmtree(scratch, ignore_errors=True)
            raise


    def cleanup_acquired_local_input(
        raw: str, local_path: str, storage_config: Any = None
    ) -> None:
        """Delete a remote scratch dir produced by ``acquire_local_input``.

        Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
        into a fresh ``rally-input-*`` directory; deleting that directory is safe once the
caller has
        finished hashing/validating/stitching/segmenting the returned file.
        """
        try:
            if input_is_local(raw, storage_config):
                return
        except ValueError:
            return
        if not local_path:
            return
        scratch = os.path.dirname(os.path.abspath(local_path))
        if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
            shutil.rmtree(scratch, ignore_errors=True)


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage

            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage

            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage

            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
```

```
138
139             return GDriveStorage
140         if name == "gdrive-api":
141             from backend.storage.gdrive_api import GDriveApiStorage
142
143             return GDriveApiStorage
144         raise ValueError(
145             f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146         )
147
148
149     def get_storage(
150         backend: Optional[str] = None,
151         *,
152         config: Optional[Dict[str, Any]] = None,
153         **overrides,
154     ) -> StorageBackend:
155         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
156         the implementation; its per-backend settings come from ``config[backend]`` merged
with
157         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
158         cfg = config or {}
159         name = backend or cfg.get("backend", "local")
160         # Per-backend settings live under the backend name; tolerate the hyphen?underscore
form so a
161         # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
hyphen
162         # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
163         settings: Dict[str, Any] = dict(
164             cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
165         )
166         settings.update(overrides)
167         return _backend_class(name)(**settings)
168
169
170     def fetch(
171         uri: str,
172         dst: "str | os.PathLike[str] | None" = None,
173         *,
174         config: Optional[Dict[str, Any]] = None,
175         overwrite: bool = False,
176     ) -> str:
177         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178         (no copy); for s3/gcs it downloads to ``dst``."""
179         ref = parse_uri(uri)
180         return get_storage(ref.backend, config=config).fetch_to_local(
181             ref, dst, overwrite=overwrite
182         )
183
184
185     def put(
186         local_path: "str | os.PathLike[str]",
187         uri: str,
188         *,
189         config: Optional[Dict[str, Any]] = None,
190         content_type: Optional[str] = None,
191     ) -> str:
192         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193         ref = parse_uri(uri)
194         get_storage(ref.backend, config=config).put(
195             local_path, ref, content_type=content_type
196         )
```

```
197         return uri
198
199
200     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
201         """True if the object at ``uri`` exists (a bucket object or a local file)."""
202         ref = parse_uri(uri)
203         return bool(get_storage(ref.backend, config=config).exists(ref))
204
205
206     def signed_url(
```

24. assistant (2026-07-08T06:29:39.930Z)

Let me check how upload object paths are formed (content-addressed vs mutable) and read the design doc's integrity argument.

25. user (2026-07-08T06:29:45.077Z)

```
9:* a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10:  through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
15:  ``input_root``) plus the streamed md5 as ``video_id``.
31:import uuid
40:    resolve_output_dir,  # P0a: app-home-aware path resolution
44:from backend.utils.paths import safe_output_filename
61:def _upload_cfg():
80:def upload_caps(cfg) -> Dict[str, int]:
98:    (storage.input_backend + input_root), so a staged upload is submittable verbatim.
106:    input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107:    if input_backend == "local" or not input_root:
110:    return {"root": input_root, "config": cfg}
113:def _abort_upload(upload_id: str) -> None:
134:        os.remove(st["tmp_path"])
139:def _normalize_upload_filename(filename: str) -> str:
150:def upload_init(req: UploadInitRequest):
157:    ext = os.path.splitext(name)[1].lower().lstrip(".")
168:    upload_id = uuid.uuid4().hex
185:                    "hasher": hashlib.md5(),
201:    tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
202:    open(tmp_path, "wb").close()
206:            "tmp_path": tmp_path,
218:async def upload_part(upload_id: str, index: int, request: Request):
256:        with open(st["tmp_path"], "ab") as fh:
287:def upload_complete(upload_id: str, req: UploadCompleteRequest):
288:    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
300:        # Remote staging: close() finalizes the streamed object; the md5 accumulated per
315:            "path": st["stage_uri"],
318:            "next": "POST /api/process with this path",
320:    final_path = os.path.join(upload_dir, st["filename"])
321:    if os.path.exists(
322:        final_path
324:        stem, ext = os.path.splitext(st["filename"])
325:        final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326:    os.replace(st["tmp_path"], final_path)
329:    video_id = compute_video_id(final_path)
331:        "path": os.path.abspath(final_path),
333:        "size_bytes": os.path.getsize(final_path),
334:        "next": "POST /api/process with this path",
339:def upload_abort(upload_id: str):
```

26. assistant (2026-07-08T06:30:13.625Z)

Let me read the design doc's integrity/trust argument and confirm no other seam.

27. user (2026-07-08T06:30:20.372Z)

```
7:> **Honesty note:** each claim is tagged `[verified]` (checked against code/runlog),
`[researched]` (needs sign-off / external pricing I could not fetch live), or `[design]`
(proposed). Not legal/financial advice.
15:The cheaper, more durable, already-ratified path is the opposite: **keep the upload streaming
to the cheap bucket** (the #480 design ? the durable copy exists *before* the L4 ever starts),
and **collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** ? which is
exactly what **#521's `phase_placement` already wires as a config flip**. The residual "~30 s
pre-validation gap" is then killed not by moving the *upload* but by the control-plane **never
downloading the file at all** (it delegates validation to the L4 per #521, and trusts the
already-known submit-time md5 per this epic's win #2 ? no re-hash). Net data movement drops from
3× to the theoretical floor: **1 client upload + 1 LAN-fast intra-region pull**, with **zero
idle-GPU cost** and **zero exposure window**.
17:**Actionable now (this PR):** the design doc + **win #2** (drop the redundant control-plane
re-hash, config-guarded, reversible). **Win #1** (validation-on-L4) is **delivered by #521** ?
this doc does not duplicate its knob. A persistent GPU (GCE VM / Cloud Run Service-with-GPU) is
**rejected at current volume** on a quantified cost basis and left as an owner-gated future ADR.
21:**Why now.** Tracing the live CUJ (`runlogs/DEPLOY_REPORT_cloud-serving_validation-
latency_2026-07-07.md` §9 `[verified]`) produced two findings:
23:1. **Data is moved 3× per run** `[verified]`: (a) upload streams straight to
`gs://?/uploads/?` (never touches control-plane disk ? the #480/#471 OOM fix,
`backend/api/uploads.py`), computing the md5 incrementally as it streams; (b) the control-plane
re-downloads the whole clip to scratch to validate ? `_execute_processing` ?
`storage.acquire_local_input(gs://?)` at `backend/orchestration.py:748`, then a **redundant re-
hash** `compute_video_id` at `:751`; (c) the worker downloads it a third time to detect
(`[worker] pulled ?`). The unlogged **~30 s pre-validation gap** is finding (b).
24:2. **The control-plane is the scaling wall** `[verified]`: validation (183 s) **and** stitch
(~12m55s, ~67 s/clip 4K re-encode, no NVENC) both run in-process on the 2-vCPU control-plane,
serialized by `_LOCAL_COMPUTE_LANE` ? ~16 min of heavy CPU per run ? the API saturates past ~5
concurrent users.
28:**The hard question this doc must resolve (compute topology).** "Upload directly to the L4"
needs an **inbound endpoint on the GPU box**, but the current L4 is a Cloud Run **Job** (batch,
no inbound HTTP ? `backend/compute/cloud_run.py`, dispatched per-request, `[verified]`). So the
epic forces a choice between **A. Cloud Run Service-with-GPU**, **B. GCE L4 VM**, and **C. a
hybrid** presign/redirect. §4 answers it with numbers.
36:| L1 | **Keep upload ? bucket** (streaming, #480). The durable copy exists **before** any
compute runs; the bucket *is* the "reliable co-located disk" the epic wants ? no VM needed for
durability. | this doc `[design]` + #480 `[verified]` | No exposure window; upload cost stays on
the cheap I/O path. |
37:| L2 | **Collapse validate + detect + stitch onto the scale-to-zero Cloud Run L4 Job** via
`phase_placement` (#521), not onto a persistent GPU. Honors **D7** (scale-to-zero) + **D16(a)**
(Jobs, not Services). | #521 `[verified]` + D7/D16 | The "L4-primary" vision is realized by
**config flips**, not a rearchitecture. |
38:| L3 | **Kill the ~30 s gap by *not downloading to the control-plane*, not by uploading onto
the L4.** Compose #521's validation-delegation (CP skips the validator suite) with win #2 (CP
trusts the submit-time md5 ? no re-hash). | this doc `[design]` | The CP stops pulling the full
file when validation is delegated; movement ? 3×?2×. |
41:| L6 | **Win #2 (drop the re-hash) is config-guarded + reversible**
(`deployment.trust_submit_video_id`, default on) and produces a **byte-identical `video_id`**
(the GCS `md5Hash` over the same bytes = `compute_video_id`). | win #2 `[verified]` | Zero-
regression; one-flag rollback. |
61:### 4.1 The measured pipeline (evidence base) `[verified]`
68:| Accept ? validate start | ~30 s | control-plane | no | **the gap** = CP re-download to
scratch (`acquire_local_input` + re-hash) |
79:### 4.2 Cost model (asia-southeast1, USD, Cloud Billing Catalog API, 2026-07-08) `[verified]`
rates / `[design]` per-run
85:| Cloud Run **L4 GPU** (no zonal redundancy) | **$0.00022404 / s** = $0.806/hr | Catalog
`[verified]` |
86:| Cloud Run vCPU (instance/jobs) | $0.0000216 / s | Catalog `[verified]` |
87:| Cloud Run memory | $0.0000024 / GiB·s | Catalog `[verified]` |
89:| GCE **g2-standard-4** (4 vCPU/16 GiB + L4), on-demand | **$0.872 / hr** = **~$637 / mo** |
Catalog `[verified]` |
90:| GCE g2-standard-4, **spot/preemptible** | $0.523 / hr = ~$382 / mo | Catalog `[verified]` |
92:| Balanced PD | $0.11 / GiB·mo | Catalog `[verified]` |
```

125:- Keep **upload ? bucket** (cheap, bounded-memory, durable-immediately). Keep the **scale-to-zero L4 Job**. Move **validate + stitch** onto the Job via `phase_placement` (#521). Make the CP **stop downloading** (delegate validation + trust the md5, win #2). The Job pulls the file **once** from the same-region bucket at LAN speed.
137:| Keep on the control-plane | Why it must stay | Code anchor `[verified]` |
139:| **Cloudflare-Access edge auth** (verify `Cf-Access` JWT) | D9 ? one identity system; direct-hit spoofing guard | `backend/api/auth.py`, gated at `/api/process` |
147:| **~~Full-file download + re-hash~~** | ? **eliminated** (delegate validation + trust md5, win #2) | this PR |
154:- **Failure/resume:** retry with capped exponential backoff; the mirror is **idempotent** (same object key = the md5). On worker preemption before the mirror confirms, the run is **lost** ? the job must fail with a *retryable* status so the client can re-submit (the source is still on the client). **Never** report success until the mirror is **confirmed** (object exists + size/md5 match).
164:| **P0** | *Today* ? detect on L4; validate+stitch on CP | `compute_target=cloud_run` | in-process | shipped `[verified]` |
166:| **P2a** | **Drop the CP re-hash** (trust submit-time md5) | `deployment.trust_submit_video_id` (default on) | re-hash on the drain | **this PR (win #2)** |
177:- **Network-egress "free intra-region" is `[researched]`, not live-verified** ? general web egress is blocked here, so same-region GCS?Cloud Run being un-billed-as-internet-egress rests on documented GCP behavior, not a fetched invoice. If it were *not* free, A? is *still* cheapest (it moves the fewest bytes); the conclusion is robust to this uncertainty.
178:- **Win #2 changes nothing when no trusted id is present** ? local/appliance uploads and non-GCS direct-URI submits (`submit_time_video_id` ? `None`) still re-hash. The win applies to the GCS cloud-serving path (the CUJ case).
179:- **P3 (worker validation-verdict serialization) is a real gap #521 leaves open** `[verified]`: today a worker-side validation reject surfaces as `rc?0` + a message, but the per-validator `ValidationResult` list (`result['results']` the SPA renders) and the #518 `validation_cache` warm are **not** reconstructed on the CP. Until P3, moving validation to the L4 (P2b) trades richer verdict UX + cache-warming for the CPU offload. Called out so P2b isn't flipped on blind.
181:- **The #513 asymmetry reverses under P2b** `[verified]`: worker-side validators run at model-default thresholds unless the CP forwards `validation.*` as `--set`. #521's validation-delegation path must forward those (or re-inherit the divergence that caused the #513 incident). Flagged for the #521 review, not owned here.
190:| P1 | **Win #2** ? drop the CP re-hash; trust submit-time md5; `deployment.trust_submit_video_id` (default on), reversible | ? | No | ? (this PR) | #524 |
218:**Internal code anchors `[verified]`:** `backend/api/uploads.py` (streamed md5), `backend/orchestration.py:220-239` (`submit_time_video_id`, #484) / `:741-763` (re-hash) / `_maybe_dispatch_remote`, `backend/api/jobs.py:80-153` (submit-time cache already trusts the md5), `backend/api/job_worker.py:122-193` (`_job_to_request`/`_run_claimed_job`), `backend/compute/cloud_run.py` (dispatch + bucket-poll), `backend/config/placement.py` (#521 `resolve_phase_placement`), `backend/worker/__main__.py` (worker validation + `--skip-validation`).
220:**External `[verified]` rates:** GCP Cloud Billing Catalog API (`cloudbilling.googleapis.com`, asia-southeast1, USD, 2026-07-08). **`[researched]`:** same-region GCS?Cloud Run egress treatment (not live-fetchable here).

28. assistant (2026-07-08T06:31:14.298Z)

Let me verify there's no alternate writer that could seed `jobs.video_id` with a client-controlled value, and confirm the download path has no post-fetch verification.

29. user (2026-07-08T06:31:20.651Z)

```
backend/api/jobs.py:51:        video_id=video_id,
backend/api/jobs.py:104:    outcome = claim_process_job(db, job_id, req, owner_id,
video_id=video_id)
backend/api/job_worker.py:180:                trusted_video_id=job.get("video_id") or None,
backend/api/job_worker.py:183:            db.set_job_video_id(job_id, result["video_id"])
backend/compute/cloud_run.py:310:            "cloud-run: job=%s done (video_id=%s rc=%d)",
self._job_name, video_id, rc
backend/compute/cloud_run.py:315:            video_id=video_id,
backend/eval/harness.py:78:        video_id=video_id, iou_threshold=iou_threshold,
gt_intervals=gt_intervals
```

```
backend/eval/regression.py:191:        video_id=video_id,
backend/infrastructure/database_jobs.py:122:            "UPDATE jobs SET video_id=? WHERE id =
?",
backend/main.py:707:        "annotations saved: video_id=%s rows=%d -> %s", req.video_id,
num_rows, path
backend/main.py:865:                        "[ASSETS] put failed video_id=%s dest=%s: %s",
backend/main.py:879:            "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s
owner=%s",
backend/main.py:1572:                    video_id=video_id,
backend/orchestration.py:397:                "[API] re-hydrated %d segments for video_id=%s from
%s (submit-time cache hit)",
backend/orchestration.py:702:        "[COMPUTE] cloud_run dispatched: execution=%s job=%s
region=%s out=%s video_id=%s",
backend/orchestration.py:723:    config=None, db=None, req=None, progress=None,
trusted_video_id=None
backend/orchestration.py:794:                "[API] trusting submit-time video_id=%s ? skipped
the drain re-hash (#524).",
backend/orchestration.py:836:                    "[API] Reusing cached segmentation for
video_id=%s; force=true bypasses cache.",
backend/orchestration.py:878:                        "[API] Reusing cached validation verdict
(passed=%s) for video_id=%s; "
backend/orchestration.py:887:                    "[API] validation-cache reuse failed for
video_id=%s ? re-validating.",
backend/orchestration.py:1018:            "[COMPUTE] video_id=%s requested=%s actual=%s
dispatched=%s status=%s",
backend/orchestration.py:1031:            video_id=video_id,
backend/orchestration.py:1215:            "[compile] stitching failed for video_id=%s: %s",
video_id, e, exc_info=True
backend/pipeline/segmenters/trajectory_hybrid.py:299:        csv_path =
runner.run_predict(video_path, out_dir, video_id=video_id)
backend/reporting/harvest.py:225:        video_id=video_id,
backend/reporting/__init__.py:61:            video_id=video_id,
backend/utils/workspace.py:109:    return VideoWorkspace(root=root, video_id=video_id,
prefix=prefix)
backend/worker/__main__.py:209:        "[worker] remote compute done: video_id=%s rc=%d
outputs=%s",
backend/worker/__main__.py:367:                "infer produced 0 segments for video_id=%s
(segmenter=%s, detector_impl=%s). "
backend/worker/__main__.py:383:            video_id=video_id,
backend/worker/__main__.py:619:            traj = runner.run_predict(local_video, traj_dir,
video_id=video_id)
```

30. user (2026-07-08T06:31:30.671Z)

```
765         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
766         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
767         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
768         # consumers (hash / validators / segmenter) use the resolved local path.
769         local_video = ""
770         try:
771             local_video = storage.acquire_local_input(
772                 req.video_path, getattr(config, "storage", None)
773             )
774             # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing
the whole
775             # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
776             # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the
submit-time
777             # cache probe trusts, so reusing it here is consistent, not a new trust
assumption. Gated by
778             # deployment.trust_submit_video_id (default on) and only honoured when the id is
well-formed
```

```
779             # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The
fetch above
780             # is unchanged; only the redundant hash is skipped.
781             trust_submitted = bool(
782                 getattr(
783                     getattr(config, "deployment", None), "trust_submit_video_id", True
784                 )
785             )
786             # STRICT: trust only an id in the EXACT form the drain would have produced
(32-char
787             # lowercase hex, per compute_video_id / submit_time_video_id). No normalization
? a
788             # differently-cased or malformed id is a signal something is off, so re-hash
rather than
789             # silently coerce it into the DB key.
790             candidate = trusted_video_id or ""
791             if trust_submitted and _looks_like_video_id(candidate):
792                 video_id = candidate
793                 logger.info(
794                     "[API] trusting submit-time video_id=%s ? skipped the drain re-hash
(#524).",
795                     video_id,
796                 )
797             else:
798                 video_id = _main.compute_video_id(local_video)
799         except Exception:
800             storage.cleanup_acquired_local_input(
801                 req.video_path, local_video, getattr(config, "storage", None)
802             )
803             raise
804         try:
805             existing_video = db.get_video(video_id)
806         except Exception:
807             storage.cleanup_acquired_local_input(
808                 req.video_path, local_video, getattr(config, "storage", None)
809             )
```

31. assistant (2026-07-08T06:32:26.812Z)

I have enough to complete the security/trust-boundary review. Let me confirm the two key clean facts one more time: `submit_time_video_id` returns `None` for non-gcs (line 225) and `ProcessRequest` has no `video_id` field (confirmed at main.py:258-272).

My analysis is complete. Here is my assessment of the SECURITY / TRUST BOUNDARY dimension.

Core risk (client-controlled video_id reaching the trusted path): CLEAN. I traced the full chain:
- `ProcessRequest` (main.py:258-272) has NO `video_id` field ? a client body cannot carry one.
- `submit_process_job` (jobs.py:98) sets `video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""` ? purely server-derived from the gs:// object's `md5Hash`; any client value is ignored.
- The only two writers of `jobs.video_id` are `claim_or_create_process_job(video_id=submit_time?)` and `set_job_video_id(result["video_id"])` (server-computed) ? no client seam.
- `trusted_video_id` is a keyword-only internal channel (job_worker.py:180 ? orchestration.py:723), and `_looks_like_video_id` (strict 32-char lowercase hex) rejects anything malformed. `_md5_hex` emits lowercase `raw.hex()`, so no case-mismatch false-negative.
- A client controls only `req.video_path` (which object), never the hash bytes. GCS computes `md5Hash` server-side, so a client cannot forge an arbitrary 32-hex id decoupled from real content. Confirmed: non-gcs backends return `None` (orchestration.py:225) ? s3/gdrive always re-hash.

Cross-tenant cache exposure: UNCHANGED vs today. The content-hash cache is global/content-addressed by design (`db.get_video(video_id)` / `query_play_segments(video_id)` are not tenant-scoped). The submit-time probe (#484) already trusts `submit_time_video_id` on the READ side;

win #2's delta is extending that trust to the WRITE side (outputs persisted under the id without a re-hash). That delta is real but gated behind object mutability ? see the finding.

The one actionable finding is the integrity delta (defense-in-depth), plus a doc-accuracy nit.

32. user (2026-07-08T06:32:44.593Z)

Structured output provided successfully